Writing React Component Reusability

We have just completed our first component within React, specifically the `TodoRowItem`, which serves to return a table row with a table header and two pieces of table data. It's an exciting milestone in our project!

Exploring the Structure

Within our JavaScript code, we invoke this component using the tag `TodoRowItem`. This tag refers back to our newly-created function, which you can locate within the file structure.

However, you might have noticed something intriguing: all of our data is currently static, hardcoded within the component. But fear not! React provides us with powerful tools to make our code more dynamic and reusable.

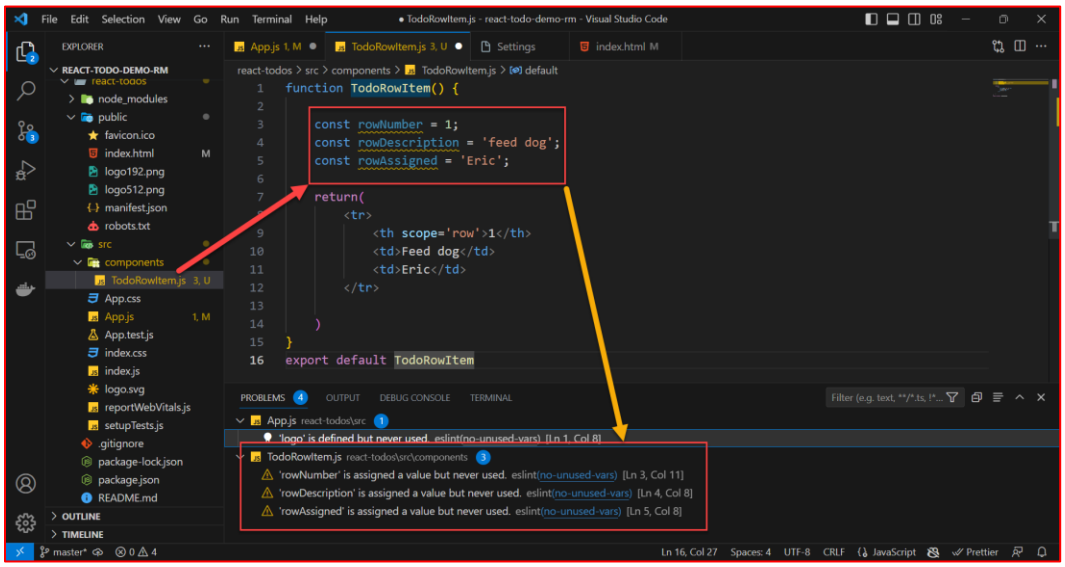
Introducing Variables

We can create variables within a component and then pass those variables into the JSON that renders the DOM. Let's see how that works:

Right underneath the `TodoRowItem` function, I'll create three variables:

- `const rowNumber = 1;`
- `const rowDescription = 'feed dog';`
- `const rowAssigned = 'Eric';`

When we save these changes, our editor may alert us that these variables are defined but not used. That's our cue to integrate them into our JSX code, the HTML that's getting rendered within the `return` statement.



Using Variables within JSX

By enclosing the variable names within curly brackets `{}` inside our JSX, we can inject the values directly. So, where our hardcoded text was, we'll replace it with the corresponding variable names:

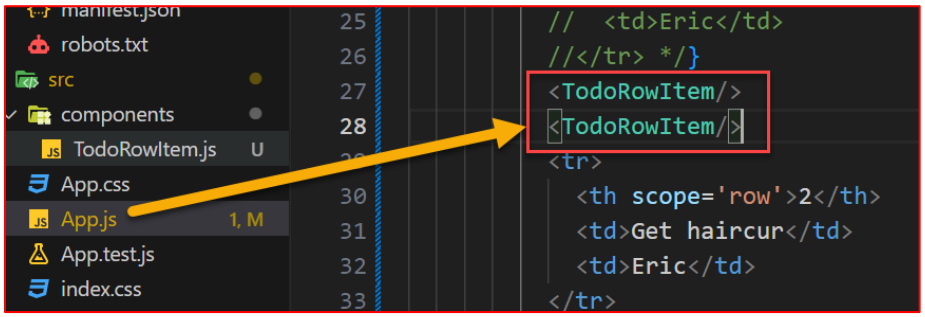
- `rowNumber` for the row number
- `rowDescription` for the description
- `rowAssigned` for the assignment



Upon saving and refreshing your browser, you might notice that nothing has visibly changed. But internally, our code is now more flexible and maintainable.

Leveraging the Power of Components

One of React's major strengths is the reusability of components. With our `TodoRowItem` component in place, you can call it multiple times, like adding another row item right under the existing one.



Going back to the browser, you'll see the repetition of "1 feed dog, Eric." This illustrates the tremendous potential of reusing components.



Identifying an Issue

Despite this progress, we're not done improving. A critical issue remains: our variables like `rowNumber`, `rowDescription`, and `rowAssigned` are still static. They don't dynamically change the `TodoRowItem`, meaning that we're not yet passing any data to influence what's being rendered on `App.js`.

Summary

This challenge sets the stage for our next topic: passing dynamic data within our components. If you're as eager as we are to see how that works, then stay tuned for our next lesson.

With React, the possibilities are vast, and we're just scratching the surface.